Guía de Testing — Restaurante Mira

Monkey test, Selenium, test de flujo, regresión y carga con Taurus.
Documento de apoyo para la presentación en Inetum.

> **Objetivo:** entender qué es cada tipo de test, para qué sirve y cómo aplicarlo a la app *restaurante-mira-frontend* (React + Vite + Firebase).

1. Visión general

Cada tipo de test responde a una pregunta distinta. Juntos forman una estrategia completa:

Test	Pregunta que responde	Herramienta
Monkey test	¿Se rompe la app si alguien la usa de forma caótica?	gremlins.js + Selenium
Selenium	(Herramienta) Controla un navegador real desde código	Selenium WebDriver
Test de flujo (E2E)	¿Puede un usuario completar una tarea de principio a fin?	Selenium + pytest
Regresión	¿Lo que funcionaba sigue funcionando tras un cambio?	Tests E2E + GitHub Actions
Carga	¿Aguanta la app con muchos usuarios a la vez?	Taurus (bzt)

2. Monkey test

Un «mono» pulsa botones, escribe texto y hace scroll **al azar** durante un tiempo. No comprueba que el resultado sea correcto: solo que la app **no se rompa** (errores en consola, pantalla en blanco, bloqueos).

> **Analogía:** dejar a un niño pequeño con tu móvil 5 minutos. Si sigue funcionando, es robusto.

Ventajas: encuentra errores que nadie habría imaginado. **Limitaciones:** no valida la lógica de negocio y los fallos a veces cuestan de reproducir (usar una semilla fija).

```
# Inyectar gremlins.js con Selenium (Python)
driver.get("http://localhost:5173")
driver.execute_script(open("gremlins.min.js").read())
driver.execute_script("gremlins.createHorde().unleash({nb: 1000})")
errores = driver.get_log("browser")   # fallo si hay SEVERE
assert not [e for e in errores if e["level"] == "SEVERE"]
```

3. Selenium

Selenium es una **herramienta**, no un tipo de test. Permite controlar un navegador real (Chrome, Firefox, Edge) desde código: abrir páginas, hacer clic, rellenar formularios y leer lo que aparece en pantalla. Simula a un usuario real.

Conceptos clave:

Concepto	Qué es

WebDriver	El «conductor» que maneja el navegador
Localizador	Cómo encontrar un elemento: By.ID, By.CSS_SELECTOR, By.XPATH
Espera explícita	WebDriverWait: esperar a que algo aparezca (clave en React, que carga asíncrono)
Headless	Ejecutar sin ventana visible (necesario en CI)

> **Consejo:** añadid atributos `data-testid` a los botones clave; así los tests no se rompen si cambia el diseño o el texto (p. ej. al traducir a ca/en).

4. Test de flujo (End-to-End)

Prueba un **recorrido completo** de usuario, tal como lo haría una persona. Se escribe normalmente con Selenium.

Flujos candidatos en Restaurante Mira:

• Búsqueda: Home → buscar «Barcelona» → aparecen cards de restaurantes.
• Detalle: clic en una card → se abre la ficha con carta y mapa.
• Reserva: elegir fecha y franja → confirmar → aparece en «Mis reservas».
• Idioma: cambiar a CA / EN → los textos cambian sin recargar.
• Login: iniciar sesión → acceso a «Mi cuenta».

```
def test_busqueda(driver):
    driver.get(BASE_URL)
    caja = driver.find_element(By.CSS_SELECTOR, "input[type=search]")
    caja.send_keys("Barcelona\n")
    cards = WebDriverWait(driver, 10).until(
        lambda d: d.find_elements(By.CSS_SELECTOR, ".card"))
    assert len(cards) > 0
```

5. Test de regresión

Regresión = algo que funcionaba deja de funcionar por un cambio nuevo. El test de regresión consiste en **volver a pasar todos los tests existentes** tras cada cambio.

> **Ejemplo real:** el commit *«iniciarSesionGoogle no pasaba a AppContent»* corrigió un fallo introducido en un merge. Un test de flujo de login con Google lo habría detectado automáticamente antes de subirlo.

Cómo aplicarlo: los tests de flujo se ejecutan automáticamente en cada push con **GitHub Actions**. Si alguno falla, el cambio no se integra. Cada bug corregido debería añadir un test nuevo para que no vuelva.

```
# .github/workflows/tests.yml (resumen)
on: [push, pull_request]
jobs:
  e2e:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run build && npx vite preview &
      - run: pip install selenium pytest && pytest tests/
```

6. Test de carga con Taurus

Taurus (`bzt`) lanza muchos **usuarios virtuales simultáneos** y mide tiempo de respuesta, errores y peticiones por segundo. Se configura con un YAML sencillo y usa JMeter por debajo.

Escalado progresivo (10 → 20 → 50): se sube poco a poco para ver dónde empieza a degradarse.

Escenario	Usuarios	Rampa	Duración	Qué observamos
Suave	10	30 s	2 min	Línea base: tiempos normales
Medio	20	1 min	3 min	¿Suben los tiempos?
Alto	50	1 min	5 min	¿Aparecen errores 5xx / timeouts?

```
execution:
- concurrency: 10     # luego 20 y 50
  ramp-up: 30s
  hold-for: 2m
  scenario: home
scenarios:
  home:
    requests:
    - http://localhost:4173/
reporting:
- module: passfail
  criteria:
  - avg-rt>2s for 30s, stop as failed
```

■ **Con cuidado:** lanzarlo siempre contra un entorno de pruebas (`vite preview` local o staging), nunca contra producción ni contra Firebase/Geoapify de pago: se consume cuota y pueden bloquear la cuenta.

7. Hallazgos de la revisión de código

#	Hallazgo	Impacto en tests
1	Imágenes hero de 1,5–2 MB	Empeora los resultados de carga → pasar a WebP
2	Bundle JS de 1,2 MB en un único fichero	Carga inicial lenta → React.lazy
3	LANG_FLAGS sin uso en Cuenta.jsx	Código muerto, eliminar
4	API key Firebase en el código	Normal en Firebase, pero restringir dominio y revisar reglas
5	Sin tests ni linter en package.json	Justo lo que cubre este plan

8. Cambios de diseño pedidos

• **Color principal verde**: p. ej. **#1F7A4D** en `--primary`.
• **Cards inferiores con border-radius 10px** (ahora 24px) en `.card`.
Estos cambios son un buen caso para demostrar la regresión: tras aplicarlos, se ejecutan los tests y se comprueba que todo sigue funcionando.

9. Plan para el miércoles

1. Explicar los conceptos (este documento).
2. Demo Selenium: test de flujo de búsqueda y reserva.

3. Demo monkey test con gremlins.js.
4. Aplicar el cambio de diseño y mostrar la regresión pasando en GitHub Actions.
5. Resultados de Taurus a 10 / 20 / 50 usuarios.
6. Uso de IA: cómo se ha usado un asistente de IA para revisar el código y generar los tests.